PeerBridge — Development Handover

This document summarizes the current PeerBridge project setup, installed technologies, implemented Member 1 work, shared files, remaining modules, and Git workflow for the team.

1. Project Overview

PeerBridge is a college-specific peer-learning platform where juniors can discover and register for learning sessions, seniors can create and manage sessions, sessions can have quizzes, users can earn points and appear on leaderboards, juniors can provide mentor feedback, and study resources can be shared centrally.

2. Technology Stack

Frontend

- React
- Vite
- JavaScript
- React Router
- Axios
- CSS

Backend

- Node.js
- Express.js
- MongoDB
- Mongoose
- JWT
- bcryptjs
- CORS
- dotenv

Development Tools

- VS Code
- Git
- GitHub
- MongoDB
- MongoDB Compass
- Postman

3. Required Installation

- Node.js + npm
- MongoDB
- Git
- VS Code
- Postman

Check Node.js and npm with:

```
node -v
npm -v
```

Check Git with:

```
git --version
```

The MongoDB database used by the project is:

```
peerbridge
```

Local MongoDB connection:

```
mongodb://127.0.0.1:27017/peerbridge
```

4. Backend Packages Already Installed

Location: peerbridge/backend

- express
- mongoose
- bcryptjs
- jsonwebtoken
- dotenv
- cors
- nodemon (development dependency)

After cloning the repository, run:

```
cd backend
npm install
```

5. Frontend Packages Already Installed

Location: peerbridge/frontend

- react
- react-dom
- react-router-dom

- axios

After cloning the repository, run:

```
cd frontend
npm install
```

6. Current Project Structure

```
PeerBridge/
├── backend/
│   ├── config/
│   │   └── db.js
│   ├── controllers/
│   │   └── authController.js
│   ├── middleware/
│   │   └── auth.js
│   ├── models/
│   │   └── User.js
│   ├── routes/
│   │   └── authRoutes.js
│   ├── .env
│   ├── .gitignore
│   ├── package.json
│   ├── package-lock.json
│   └── server.js
└── frontend/
    ├── public/
    ├── src/
    │   ├── api/
    │   │   └── axios.js
    │   ├── components/
    │   │   └── ProtectedRoute.jsx
    │   ├── context/
    │   │   └── AuthContext.jsx
    │   ├── pages/
    │   │   ├── Login.jsx
    │   │   ├── Signup.jsx
    │   │   └── Profile.jsx
    │   ├── App.jsx
    │   ├── index.css
    │   └── main.jsx
    ├── .gitignore
    ├── package.json
    ├── package-lock.json
    └── vite.config.js
```

7. Member 1 — Implemented Work

User Registration

Implemented: POST /api/auth/signup

Registration fields: name, email, password, role, college, year. Roles are junior and senior.

Password Security

bcryptjs is used to hash passwords before they are stored in MongoDB. Passwords are not stored as plain text.

Login

Implemented: POST /api/auth/login. Valid credentials return a JWT token and user information.

JWT Authentication

JWT is used for authentication. The token is stored in frontend localStorage and protected requests use the Authorization: Bearer <token> header.

Authentication Middleware

File: backend/middleware/auth.js

- protect — checks whether the user has a valid JWT.
- authorize — checks whether the authenticated user has an allowed role.

User Model

File: backend/models/User.js

Fields: name, email, password, role, college, year, bio, skills, photo, points. Points start at 0.

Profile

Users can view their profile and update bio, skills, and photo.

Update endpoint: PUT /api/auth/profile

Frontend Authentication

frontend/src/context/AuthContext.jsx manages token, user, login(), logout(), and isLoggedIn.

Protected Frontend Routes

frontend/src/components/ProtectedRoute.jsx redirects unauthenticated users to /login when they try to access protected pages.

Current Frontend Pages

- /login

- /signup
- /profile

Axios Setup

File: frontend/src/api/axios.js

A central Axios instance is configured with the backend API base URL so other modules can reuse it.

8. Current API Routes

```
POST /api/auth/signup
POST /api/auth/login
GET  /api/auth/me
PUT  /api/auth/profile
```

The /me and /profile routes are protected.

9. Backend and Frontend Servers

Backend: http://localhost:5000

Frontend development server: http://localhost:5173

10. Environment Variables

The backend .env contains PORT, MONGO_URI, JWT_SECRET, and JWT_EXPIRES_IN.

IMPORTANT: .env is ignored by Git and must not be committed to GitHub. Each team member should create their own backend/.env using the team's agreed development values.

11. Shared/Core Files — Do Not Rebuild Authentication

The following files form the existing Member 1 authentication/profile foundation:

```
backend/models/User.js
backend/middleware/auth.js
backend/controllers/authController.js
backend/routes/authRoutes.js
frontend/context/AuthContext.jsx
frontend/components/ProtectedRoute.jsx
```

Members 2 and 3 should reuse the existing authentication middleware and user model rather than creating another authentication system.

12. Member 2 — Remaining Work: Session Management

Member 2 is responsible for the Session Management module.

- Create Session model
- Create session controller
- Create session routes
- Senior: create, update, delete, and view own sessions
- Senior: schedule sessions and add meeting links
- Junior: view available sessions
- Junior: search/filter sessions
- Junior: register for sessions

Suggested backend files:

```
backend/models/Session.js
backend/controllers/sessionController.js
backend/routes/sessionRoutes.js
```

13. Member 3 — Remaining Work: Quiz Module

Member 3 is responsible for the Quiz module.

- Session-linked quizzes
- Questions
- Options
- Answers
- Automatic scoring
- Quiz results

14. Later PeerBridge Modules

- Gamification: points and leaderboard
- Mentor feedback: ratings and reviews
- Resources: centralized study material

15. GitHub Repository and Branching

Repository: RovenaMadharasi/PeerBridge

Member 1 work is currently on the member-1 branch and has been committed and pushed.

Current Member 1 commit message:

```
Complete Member 1 authentication and profile module
```

Recommended team structure:

```
main       → stable project version
member-1   → authentication/profile work
member-2   → session management
member-3   → quiz management
```

Do not directly develop experimental work on main. Each member should work on their own branch and later merge tested work into main.

16. New Member Setup

After accepting the GitHub collaborator invitation:

```
git clone <PeerBridge-repository-url>
cd PeerBridge
```

Install backend dependencies:

```
cd backend
npm install
```

Install frontend dependencies:

```
cd ../frontend
npm install
```

Create backend/.env locally and make sure MongoDB is running.

17. Team Rules

- Do not commit .env.
- Do not commit node_modules.
- Reuse the existing authentication system.
- Do not modify Member 1 authentication files unnecessarily.
- Work on separate branches for separate modules.
- Test a module before merging it into main.
- Keep main as the stable version.

18. Current Status

Member 1 authentication and profile work is complete and committed to GitHub on member-1. The next development work belongs to the other assigned members: Member 2 handles sessions and Member 3 handles quizzes.